



EC8206 - Functional Programming

Project Report:

A Type Safe Arithmetic Expression Interpreter in Haskell

Department of Electrical and Information Engineering

Faculty of Engineering

University of Ruhuna

EG/2021/4748-RASHMI E.D.K

1. Project Overview

The interpreter is in `src/Expr.hs`; `app/Main.hs` runs the demonstrations.

Part A: `Expr` has `Lit Double`, `Var String`, `Add`, `Sub`, `Mul`, `Div` and the extension constructor `Let String Expr Expr` (meaning `let x = e1 in e2`). `Env` is `[(String, Double)]`.

Part B: `eval :: Env -> Expr -> Either String Double` has one equation per constructor; division by zero and undefined variables are returned as `Left`. [1]

Part C: `simplify` applies the identities $x+0$, $0+x$, $x-0$, $x\times 1$, $1\times x$ and $x/1$; `evalBatch` (`rights . map (eval env)`) keeps successful results, and `batchReport` uses a single `foldr` to count (successes, failures). `eval env` is a partial application, so `map (eval env) exprs` covers the currying requirement.

Requirements: GHC (tested with 9.4.7, no extra packages), Python 3 for the Part D snippet. From the project folder:

```
ghc -isrc -Wall -outputdir build -o interpreter app/Main.hs # compiles with no warnings
./interpreter          # on Windows PowerShell: .\interpreter.exe
ghci -isrc app/Main.hs      # interactive: eval [("x",5)] (Add (Var "x") (Lit 2))
python eval/eval_imperative.py # imperative version (Part D)
```

2. Sample Evaluations

Environment: `[( "x",5), ( "y",2)]`. `Main.hs` prints the expected value next to the actual value of `eval`, and flags any mismatch. All six pass.

No	Expression	Expected	Actual	Result
1	$(2 + (3 * 4))$	Right 14.0	Right 14.0	PASS
2	$(x - y)$	Right 3.0	Right 3.0	PASS
3	$(let\ z = 10\ in\ (z / y))$	Right 5.0	Right 5.0	PASS
4	$(1 / (x - 5))$	Left "Division by zero"	Left "Division by zero"	PASS
5	$(x + w)$	Left "Undefined variable: w"	Left "Undefined variable: w"	PASS
6	$(let\ x = 1\ in\ (x + y))$	Right 3.0	Right 3.0	PASS

batch results :

Input	simplify output	Rule applied
$((x * 1) + 0)$	x	$x\times 1 \rightarrow x$, then $x+0 \rightarrow x$
$(x * 1)$	x	$x\times 1 \rightarrow x$
$((y / 1) - 0)$	y	$y/1 \rightarrow y$, then $y-0 \rightarrow y$
$((1 / 0) * 0)$	$((1 / 0) * 0)$ (unchanged)	$x\times 0 \rightarrow 0$ deliberately not applied

Over the six sample expressions, `evalBatch env0 exprs` returns `[14.0, 3.0, 5.0, 3.0]` and `batchReport env0 exprs` returns `(4, 2)`: four succeeded (samples 1, 2, 3, 6) and two failed (samples 4 and 5).

3. Part D - Comparative Analysis

Imperative version (eval/eval_imperative.py). Nodes are tuples, the environment is a mutable dictionary, an if/elif chain acts as the switch, and errors are exceptions. [2]

```
def evaluate(node, env):
    kind = node[0]
    if kind == "lit":
        return node[1]
    elif kind == "var":
        return env[node[1]]          # KeyError if undefined (found at run time)
    else:
        result = evaluate(node[1], env)
        right = evaluate(node[2], env)
        if kind == "add": result = result + right
        elif kind == "sub": result = result - right
        elif kind == "mul": result = result * right
        elif kind == "div":
            if right == 0: raise ZeroDivisionError("Division by zero")
            result = result / right
        else: raise ValueError("unknown node: " + kind)
    return result
```

Functional version (core of eval in src/Expr.hs):

```
eval :: Env -> Expr -> Either String Double
eval _ (Lit n) = Right n
eval env (Var x) = case lookup x env of
    Nothing -> Left ("Undefined variable: " ++ x)
    Just v   -> Right v
eval env (Div a b) = do x <- eval env a
    y <- eval env b
    if y == 0 then Left "Division by zero" else return (x / y)
eval env (Let name bound body) = do v <- eval env bound
    eval ((name, v) : env) body -- Add, Sub, Mul similar
```

Error representation and propagation

In Haskell the possibility of failure is written in the type: eval returns Either String Double, so a caller cannot get at the Double without handling the Left case. Inside eval, the do blocks in the Either monad stop at the first Left and pass it upwards, so no explicit error checks are needed after each recursive call. Errors are ordinary values: batchReport counts them with foldr, and Main compares them with ==. In Python the errors are exceptions (KeyError, ZeroDivisionError) that do not appear in the function signature, so a caller can forget to catch them and the program only fails when that input arrives.

Mutable state

The Python version relies on a mutable dictionary env that is shared by every call, and on the variable result, which is reassigned inside the if/elif chain. Adding a Let would mean writing into env and then remembering to restore or delete the binding afterwards, otherwise it would leak out of its scope. In Haskell nothing is modified: Let builds a new list (name, v) : env and passes it only to the body, so the caller's environment is untouched and shadowing works automatically (sample 6).

Bugs the type checker rules out before the program runs

(1) A malformed tree such as ("add", ("lit", 1)) is a legal Python value that crashes with an IndexError at run time, whereas Add (Lit 1) does not type-check.

(2) A misspelt tag like "mull" reaches the else branch at run time; a misspelt constructor is a compile error. (3) Putting a string in a literal, ("lit", "abc"), fails only when it is added to something; Lit "abc" is rejected because Lit holds a Double.

(4) Using the result of eval as a plain number, e.g. eval env e + 1, is a type error, because the result is an Either. (5) With -Wall, GHC warns if eval forgets a constructor.

Limitations

Python is more familiar and simpler to read for most programmers, while Haskell requires understanding Either and the do notation. Neither language can detect division by zero or an unbound variable statically, since they depend on run-time values; this is exactly why eval returns Either.

4. Part E - Reflection

Pure functions

eval env e depends only on its two arguments: it reads no global state, performs no input/output and changes nothing, so the same env and e always produce the same Either value. This is what makes testing simple: in Main.hs, each sample is just show (eval env0 expr) compared with a string, with no setup or reset between cases. simplify is also pure, and each rule is an equation such as simplify (Sub a b) returning e when the simplified right-hand side is Lit 0, so I can reason about it by substitution. That reasoning showed a real problem with the textbook rule $x \times 0 \rightarrow 0$: for $(1/0) * 0$, eval gives Left "Division by zero" but the rewritten Lit 0 would give Right 0.0, so the rewrite would change the program's meaning. I therefore removed that rule, and the bad*0 line in the demo shows the expression is left unchanged.

Immutability

No function modifies an Expr or an Env. simplify returns a new tree and the original expression is still intact, which is why the demo can print the expression before and after simplification. Let does not overwrite x; it conses a new pair onto the front of the list, and because lookup returns the first match, the inner binding shadows the outer one only while the body is being evaluated. Once eval returns, the caller still holds its old Env.

Why the shape of `Expr` rules out invalid expressions

Expr is a sum type, and each constructor fixes the number and type of its children. There is no Add with one operand, no operator stored as a string that could be misspelt, and no Lit "abc", because a literal holds a Double. Some invalid expressions can still be built, namely Var "w" with no binding and Div e (Lit 0), but these depend on run-time values; they are not rejected by the compiler and instead are caught by pattern matching in eval, which returns a Left that every caller must handle. Adding a new constructor to Expr makes -Wall flag every function that does not yet handle it (for example eval and pretty). simplify is an exception, because it ends with a catch-all equation simplify e = e for Lit and Var; I accepted this trade-off for brevity, but it means a new constructor would be silently left unsimplified.

5. References

- [1] G.Hutton, Programming in Haskell, 2nd ed., Cambridge University Press, 2016.
- [2] "Haskell.org," 2014. [Online]. Available: <https://www.haskell.org/>. [Accessed 2026/10/01].